3.1.1. Numpy Array Operations

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, r and c, representing the number of rows and the number of columns.
- The next r lines contain c space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases

```python
import numpy as np

import numpy as np

# Input rows and columns
r, c = map(int, input().split())

# Read elements
elements = []
for _ in range(r):
    elements.extend(map(int, input().split()))

# Create NumPy array and reshape
arr = np.array(elements).reshape(r, c)

# Output
print(arr)
print(arr.ndim)
print(arr.shape)
print(arr.size)
```

3.2.1. Numpy: Matrix Operations

The given code takes two 3×3 matrices, $\mathrm{matrix\_a}$, and $\mathrm{matrix\_b}$, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition** $(\mathrm{matrix\_a} + \mathrm{matrix\_b})$
2. **Subtraction** $(\mathrm{matrix\_a} - \mathrm{matrix\_b})$
3. **Element-wise Multiplication** $(\mathrm{matrix\_a} * \mathrm{matrix\_b})$
4. **Matrix Multiplication** $(\mathrm{matrix\_a} \cdot \mathrm{matrix\_b})$
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for $\mathrm{matrix\_a}$, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for $\mathrm{matrix\_b}$, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.

matrixOp...

```
#  import numpy as np

# # Input matrices
# print("Enter Matrix A:")
# matrix_a = np.array([list(map(int, input().split())) for i in range(3)])

# print("Enter Matrix B:")
# matrix_b = np.array([list(map(int, input().split())) for i in range(3)])

# # Addition
# print("Addition (A + B):")
# print(matrix_a + matrix_b)
# # Subtraction
# print("Subtraction (A - B):")
# print(matrix)
# # Multiplication (element-wise)
# print("Element-wise Multiplication (A * B):")
# print(matrix_a + matrix_b)
# # Matrix multiplication (dot product)
# print("A dot B:")

# # Transpose
# print("Transpose of A:")
```

Terminal Test cases

< Prev Reset Submit Next >

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking**: Stack the two matrices horizontally (side by side).
- **Vertical Stacking**: Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases +

stacking.py

```python
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)
ar1=np.hstack((arr1,arr2))
print("Horizontal Stack:")
print(ar1)
# Perform vertical stacking (vstack)
ar2=np.vstack((arr1,arr2))
print("Vertical Stack:")
print(ar2)
```

codetantra - Yahoo India Search | Course | i love pdf - Yahoo India Search | Download file | iLovePDF

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774ee9df4ab787...

CODETANTRA Home Learn Anywhere 202501090153@mitaoe.ac.in Support Logout

Essentials of Data Science Laboratory - 2304102L - 2026

Search course

- 3. Practical 3
  - 3.1. Practice Lab Assignment
    - 3.1.1. Numpy Array Operations
  - 3.2. Lab Assignment
    - 3.2.1. Numpy: Matrix Operations
    - 3.2.2. Numpy: Horizontal and Vertical Stac...
    - 3.2.3. Numpy: Custom Sequence Generation
    - 3.2.4. Numpy: Arithmetic and Statistical Op...
    - 3.2.5. Numpy: Copying and Viewing Arrays
    - 3.2.6. Numpy: Searching, Sorting, Countin...
    - 3.2.7. Student Data Analysis and Operations
- 4. Practical 4
- 5. Practical 5

3.2.3. Numpy: Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases +

customS...

```python
import numpy as np

# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())

# Generate the sequence using np.arange()
sq=np.arange(start,stop,step)
# Print the generated sequence
print(sq)
```

Terminal | Test cases

< Prev | Reset | Submit | Next >

25°C Clear | Search | ENG IN | 04:51 10-05-2026

codetantra - Yahoo India Search | Course | i love pdf - Yahoo India Search | Download file | iLovePDF

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774ef2cf4ab787...

CODETANTRA Home Learn Anywhere 202501090153@mitaoe.ac.in Support Logout

Essentials of Data Science Laboratory - 2304102L - 2026

Search course

- 3. Practical 3
  - 3.1. Practice Lab Assignment
    - 3.1.1. Numpy Array Operations
  - 3.2. Lab Assignment
    - 3.2.1. Numpy: Matrix Operations
    - 3.2.2. Numpy: Horizontal and Vertical Stac...
    - 3.2.3. Numpy: Custom Sequence Generation
    - 3.2.4. Numpy: Arithmetic and Statistical Op...
    - 3.2.5. Numpy: Copying and Viewing Arrays
    - 3.2.6. Numpy: Searching, Sorting, Countin...
    - 3.2.7. Student Data Analysis and Operations
- 4. Practical 4
- 5. Practical 5

3.2.4. Numpy: Arithmetic and Statistical Operations, M... 01:17

You are given two arrays `A` and `B`. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. **Arithmetic Operations**:
   - Compute the element-wise sum, difference, and product of the two arrays.
2. **Statistical Operations**:
   - Calculate the mean, median, and standard deviation of array `A`.
3. **Bitwise Operations**:
   - Perform bitwise `AND`, bitwise `OR`, and bitwise `XOR` on the arrays (ex: A_i OR B_i).

Input Format:
- The first line contains space-separated integers representing the elements of array `A`.
- The second line contains space-separated integers representing the elements of array `B`.

Output Format:
- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases +

different... Submit

```python
# import numpy as np

# def array_operations(A, B):

#     # Convert A and B to NumPy arrays

#     # Arithmetic Operations
#     sum_result =
#     diff_result =
#     prod_result =

#     # Statistical Operations
#     mean_A =
#     median_A =
#     std_dev_A =

#     # Bitwise Operations
#     and_result =
#     or_result =
#     xor_result =

#     # Output results with one space between each element
#     print("Element-wise Sum:", ' '.join(map(str, sum_result)))
```

Terminal Test cases

< Prev Reset Submit Next >

25°C Clear Search ENG IN 04:51 10-05-2026

codetantra - Yahoo India Search | Course | i love pdf - Yahoo India Search | Download file | iLovePDF

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774efaaf4ab787...

CODETANTRA Home Learn Anywhere 202501090153@mitaoe.ac.in Support Logout

Essentials of Data Science Laboratory - 2304102L - 2026

Search course ctrl + k

- 3. Practical 3
  - 3.1. Practice Lab Assignment
    - 3.1.1. Numpy Array Operations
  - 3.2. Lab Assignment
    - 3.2.1. Numpy: Matrix Operations
    - 3.2.2. Numpy: Horizontal and Vertical Stac...
    - 3.2.3. Numpy: Custom Sequence Generation
    - 3.2.4. Numpy: Arithmetic and Statistical Op...
    - 3.2.5. Numpy: Copying and Viewing Arrays
    - 3.2.6. Numpy: Searching, Sorting, Countin...
    - 3.2.7. Student Data Analysis and Operations
- 4. Practical 4
- 5. Practical 5

3.2.5. Numpy: Copying and Viewing Arrays

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

```
Original array after modifying view: <original_array>
View array: <view_array>
```

- After modifying the copy:

```
Original array after modifying copy: <original_array>
Copy array: <copy_array>
```

copyAnd... Submit

```
# import numpy as np

# inputlist = list(map(int,input().split(" ")))

# # Original array
# original_array = np.array(inputlist)

# # Create a view
# view_array =

# # Create a copy
# copy_array =

# # Modify the view
# view_array[0] = 99
# print("Original array after modifying view:", original_array)
# print("View array:", view_array)

# # Modify the copy
# copy_array[1] = 88
# print("Original array after modifying copy:", original_array)
# print("Copy array:", copy_array)
import numpy as np
```

Terminal Test cases

< Prev Reset Submit Next >

25°C Clear Search ENG IN 04:51 10-05-2026

codetantra - Yahoo India Search | Course | i love pdf - Yahoo India Search | Download file | iLovePDF

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e#/contents/698c54860aba96214c8b7d65/698c54860aba96214c8b7d67/6774f064f4ab787...

CODETANTRA Home Learn Anywhere 202501090153@mitaoe.ac.in Support Logout

Essentials of Data Science Laboratory - 2304102L - 2026

Search course

- 3. Practical 3
  - 3.1. Practice Lab Assignment
    - 3.1.1. Numpy Array Operations
  - 3.2. Lab Assignment
    - 3.2.1. Numpy: Matrix Operations
    - 3.2.2. Numpy: Horizontal and Vertical Stac...
    - 3.2.3. Numpy: Custom Sequence Generation
    - 3.2.4. Numpy: Arithmetic and Statistical Op...
    - 3.2.5. Numpy: Copying and Viewing Arrays
    - 3.2.6. Numpy: Searching, Sorting, Countin...
    - 3.2.7. Student Data Analysis and Operations
- 4. Practical 4
- 5. Practical 5

3.2.6. Numpy: Searching, Sorting, Counting, Broadcas... 02:02

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user.
Additionally, it takes the following inputs:

- `search_value`: The value to search for in the array.
- `count_value`: The value to count its occurrences in the array.
- `broadcast_value`: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. Searching: Find the indices where `search_value` appears in `array1` and print these indices.
2. Counting: Count how many times `count_value` appears in `array1` and print the count.
3. Broadcasting: Add `broadcast_value` to each element of `array1` using broadcasting, and print the resulting array.
4. Sorting: Sort `array1` in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing `array1`.
2. An integer `search_value` represents the value to search for in the array.
3. An integer `count_value` represents the value to count in the array.
4. An integer `broadcast_value` represents the value to add to each element of the array.

arrayOpe... Submit

```
# import numpy as np

# # Input array from the user
# array1 = np.array(list(map(int, input().split())))

# # Searching
# search_value = int(input("Value to search: "))
# count_value = int(input("Value to count: "))
# broadcast_value = int(input("Value to add: "))

# # Find indices where value matches in array1

# # Count occurrences in array1

# # Broadcasting addition

# # Sort the first array
import numpy as np

# Input array from the user
array1 = np.array(list(map(int, input().split())))

# Searching
search_value = int(input("Value to search: "))
```

Terminal Test cases

< Prev Reset Submit Next >

25°C Clear Search ENG IN 04:52 10-05-2026

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.
- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the

Operatio... Submit

```python
# import numpy as np

# a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

# # 1. Print all student details
# print("All student Details:\n", )

# # 2. print total students

# print("Total Students:", )

# # 3. Print all student Roll numbers
# print("All Student Roll Nos", )

# # 4. Print subject 1 marks
# print("Subject 1 Marks", )

# # 5. print minimum marks of Subject 2
# print("Min marks in Subject 2", )

# # 6. print maximum marks of Subject 3
# print("Max marks in Subject 3", )

# # 7. Print All subject marks
```

Terminal Test cases

< Prev Reset Submit Next >